Radar Telemetry Over LoRa

1. Project Charter

This project came from work, where the radar team has limited access to the ground control station during integration events and flight testing. Because of that problem, I will build a simple LoRa telemetry system that gives us another way to gather status data during testing. I will use one Heltec board as the air Heltec and a second Heltec board as the ground Heltec. We will use a laptop as the air the radar simulator, and it will send mode commands to a Heltec, which will act as the air Heltec, through USB serial communication. The air Heltec will read those commands, add timing and health fields, and send short telemetry packets over the LoRa. The ground Heltec will capture the packets and forward them to the ground laptop, where we will display and log the newest status values.

We will simulate radar behavior instead of using actual radar hardware because the hardware is not available for this project. The simulated system will support booting, standby, active, sleep, and error modes during the final demonstration. The operator will be able to change those modes from the air laptop while the air Heltec continues sending updated status packets. The main telemetry fields will be mode, boot time, run time, sleep state, health status, and alarm status. I will keep the packet format simple and readable so we can debug quickly and test often during the quarter. This keeps the technical work focused on communication, monitoring, and system reliability instead of difficult hardware access problems.

We will build the project in stages so each subsystem can be tested before full integration begins. First, we will confirm serial communication between the air laptop and the air Heltec under

controlled bench conditions. Next, we will confirm packet creation and LoRa transmission from the air Heltec to the ground Heltec. After that, we will confirm forwarding from the ground Heltec to the ground laptop through USB serial communication. Last, we will confirm live display and basic logging on the ground laptop during one end-to-end test. This staged approach lowers risk because we can find problems early and fix them before combining the full system.

Our minimum viable product will be a stable bench demonstration that shows command input, packet generation, wireless transmission, receiver forwarding, and live display. We will consider the minimum viable product complete when one continuous test runs without resets and shows matching data on the ground laptop. The air Heltec must send packets at a fixed interval, with a target range of one to five seconds between updates. The ground side must display matching information clearly enough for an observer to understand the current system condition. We will use a simple Python display first because it is faster to build and easier to debug than a more advanced dashboard. This choice keeps the project realistic and gives us a better chance of finishing on time.

If the core system works early, we will add packet counters, saved logs, and simple data export for later review. If time still remains, we will do walking tests or slow car tests to simulate motion during integration events. These motion tests will be helpful, but they are not part of the promised minimum product. The main promise for this quarter is a working end-to-end telemetry system that can be shown clearly during a bench demonstration. This project is realistic for eight weeks because the hardware count is small, the packet format is simple, and testing can stay indoors.

The biggest project risks are time limits, LoRa setup problems, serial communication errors, and delays from building a dashboard that is too complex. We will reduce those risks by keeping the

packets short, using a simple Python display, and removing lower-priority features if milestones slip. Another risk is that the air and ground systems may not match during early testing because of timing or format errors. We will reduce that risk by testing each step separately before the full radio link is used. We already have the main hardware and software tools, which lowers cost and lowers setup risk. This combination gives us a strong chance of finishing the project on time.

2. Group Management

I will serve as the primary project lead and will handle the main air Heltec, ground Heltec, and integration responsibilities for the project. Bryce Rooney will serve as a consultant and peer mentor during the early part of the quarter because he is working on a separate project. I will use Bryce as a technical sounding board, peer reviewer, and secondary tester whenever Bryce has time available to help. If Bryce later decides that his separate project cannot continue because of classified work restrictions, he will join this project full time. If that happens, Bryce will move from consultant support into full development, testing, documentation, and final presentation work.

We will make major decisions through discussion and simple agreement, with me making the final call if a quick schedule decision becomes necessary. We will communicate through Discord, email, and shared Google Docs because those tools worked well in previous classes. We will also use GitHub to store code, track working versions, and collect milestone evidence during the quarter. These tools will make it easier to keep notes, share files, and stay organized across the full schedule.

We will know the project is off schedule when a weekly milestone cannot be demonstrated with code, logs, screenshots, or video evidence. If a milestone slips by more than one week, we will

remove lower-priority work and return focus to the minimum viable product. I will be responsible for the main project milestones, while Bryce will support review, testing, and added development work when available. This structure keeps leadership clear while still allowing shared technical discussion and support.

3. Project Development

The main development roles are air-side firmware, ground-side firmware, display software, testing, and project documentation. The project will use two Heltec WiFi LoRa boards, two laptops, USB cables, PlatformIO, VS Code, Python, GitHub, and shared documents. We already have the main hardware, so the required project can be completed without major new purchases. A battery pack may be useful after the minimum viable product, but it is not required for the promised bench demonstration.

The air Heltec and ground Heltec firmware will be developed in C++ using PlatformIO inside VS Code. The air laptop control script and ground laptop display will be developed in Python because that supports fast serial testing and easy display changes. If additional data plotting becomes useful later, we will export logs to CSV or Excel and view them through Power BI. That later step is optional and will only happen if the required system is already stable.

Testing will happen in layers so each subsystem can be verified before the full system is integrated. The first tests will confirm USB serial input from the air laptop to the air Heltec under stable bench conditions. The next tests will confirm LoRa transmission from the air Heltec to the ground Heltec at short indoor range. The next tests will confirm forwarding from the ground Heltec to the ground laptop through USB serial communication. The final tests will confirm live display, saved logs, and stable operation during one complete end-to-end demonstration.

Documentation will include code comments, setup notes, packet examples, screenshots, test logs, and short videos for each major deliverable. This documentation will make milestone reviews clearer and reduce confusion during final report preparation and presentation work. Each major deliverable will include visible proof that the system works instead of only written progress claims. That evidence will help us stay organized and respond quickly if schedule problems appear.

4. Project Milestones and Schedule

The first deliverable will be a finished system design with packet format, telemetry fields, role assignments, and a realistic eight-week schedule. The second deliverable will be a working air-side transmitter that accepts serial commands and generates readable telemetry packets. The third deliverable will be a working LoRa link that sends packets from the air Heltec and receives matching packets on the ground side. The fourth deliverable will be a working ground display that shows live telemetry values and saves basic logs. The fifth deliverable will be a complete minimum viable product demonstration that proves the full telemetry path works from command input to final display.

Week one will finalize scope, assign roles, confirm hardware access, and create the repository structure for firmware and display work. Week two will define telemetry fields, packet format, software interfaces, and the exact success criteria for the minimum viable product. Week three will complete serial communication tests and command handling from the air laptop to the air Heltec. Week four will complete telemetry packet creation and short-range LoRa transmission from the air Heltec to the ground Heltec.

Week five will stabilize forwarding on the ground side, confirm matching packet output, and save basic logs that show repeated packet exchange. Week six will build the Python display on the ground laptop and show live values for mode, alarm, health, boot time, and run time. Week seven will focus on integration testing, bug fixing, demonstration practice, and collecting screenshots, logs, and short videos. Week eight will finish documentation, finalize the presentation, and perform the complete bench demonstration of the telemetry system.

Each deliverable will require visible proof such as a screenshot, short video, packet log, or documented GitHub update. Deliverable one will be shown through the finished design document and packet format description. Deliverable two will be shown through serial output proving the air Heltec is receiving and packaging commands correctly. Deliverable three will be shown through matching air-side and ground-side packet logs from a live bench test. Deliverable four will be shown through a working display window and saved telemetry output files. Deliverable five will be shown through one continuous end-to-end demonstration video and final system screenshots.

The highest-priority milestones are serial command input, telemetry packet creation, LoRa packet transfer, receiver forwarding, and ground display. Useful but lower-priority work includes cleaner graphics, packet counters, connection status indicators, and CSV export for later Power BI use. Hopeful work, only if time permits, includes walking tests or slow car tests that simulate UAV taxi or motion during transmission. This schedule remains realistic because all advanced features are kept outside the required minimum product.

5. Excel-Ready Schedule

Weekly Milestones Schedule

Week	Milestone	Owner	Proof of Completion
1	Finalize scope, roles, and repository setup	Manny	Repository and task list
2	Complete system design and packet format	Manny	Design document
3	Complete air laptop to air Heltec serial test	Manny	Serial output screenshot
4	Complete telemetry packet creation and LoRa send	Manny	Sent packet log
5	Complete ground receive and forwarding	Manny	Received packet log
6	Complete ground laptop Python display	Manny	Live display screenshot
7	Complete integration testing and bug fixes	Manny	End-to-end test notes
8	Complete final demo and report	Manny	Demo video and report
1 to 8	Review, feedback, and support as available	Bryce	Review notes and test support if his project is not sufficient

Gantt Chart Schedule

Task	W1	W2	W3	W4	W5	W6	W7	W8
Scope and setup	■							
Design and packet format		■						
Serial command test			■					
Packet creation and LoRa send				■				
Ground receive and forwarding					■			
Ground display						■		

Integration and fixes	■
Final demo and report	■

The highest priority milestones are serial command input, telemetry packet creation, LoRa transfer, receiver forwarding, and ground display. Optional work such as CSV export, Power BI plots, and motion testing will only happen after the MVP is stable.